

Foundations of Programming Languages

Exercises 2

Typed Arithmetic Expressions

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

The exercise numbers in this list refer to the exercise numbers in book *Types and Programming Languages*✶. Benjamin Pierce, The MIT Press, 2002.

Pencil and paper For these exercises, justify your answers: examples must be evaluated or type-checked step by step, and proofs must cover the new cases only.

1. Write a term t and a value v such that $t \rightarrow^* v$ (Definition 3.5.9) but $t \rightarrow v$ does not hold.
2. Define multi-step evaluation as a set of inference rules (Exercise 3.5.10).
3. Variations of the E-If rule (Exercise 3.5.13). For each variation, say whether the evaluation relation changes (is it still deterministic? does it still reach the same normal forms?) and give an example term that shows it.
4. Extend the proof of “Determinacy of one-step evaluation” (Theorem 3.5.4) to the rules for `succ`, `pred` and `iszero` in Figure 3-2 (Exercise 3.5.14). Write only the new cases.
5. Write a well-typed term that evaluates to a normal form in at least two steps, and show its evaluation.
 - (a) Is the resulting normal form a value?
 - (b) Can a well-typed term evaluate to a normal form that is not a value? Either give an example or explain, referring to the theorems of Section 8.3, why not.
6. Write a term that evaluates to a value, yet is not well typed under the rules in Figures 8-1 and 8-2. Explain which subterm cannot be typed.
7. Subject expansion for typed arithmetic expressions (Exercise 8.3.6): is it true that if $t \rightarrow t'$ and $t' : T$ then $t : T$? Prove it or give a counterexample.

8. Consider adding a new term $\max t_1 t_2$ to the language of terms, denoting the larger of two natural numbers. Write the evaluation rules for $\max$, in the style of Figure 3-2: the computation rules and the congruence rules. Evaluation is left to right: we first evaluate t_1 and, only once it is a numeric value nv_1 , we evaluate t_2 . Terms such as $\max \text{true } 0$ are stuck; do not add rules for them.
9. Propose a typing rule for $\max t_1 t_2$, in the style of Figure 8-2.
10. Extend the proofs of the Progress Theorem (8.3.2) and of the Preservation Theorem (8.3.3) to account for $\max$, using the rules from items 8 and 9. Write only the new cases. Do we need to change the Canonical Forms Lemma (8.3.1)? Justify in one or two sentences.

Programming The following exercises can be based on the Haskell source code adapted from TAPL and provided at the course's web site.

11. Extend the `consts` function to the terms `succ`, `pred` and `iszero`.
12. Extend the `Term` datatype and the `step` function with the rules in Figure 3-2 (arithmetic expressions).
13. Write a Haskell term for the expression in item 1 and check, using `step`, that it takes more than one step to reach a value.
14. Write a function `eval :: Term → Term` that finds the normal form of a given term, using `step`. If the normal form is not a value, the function fails with a Haskell exception (non-exhaustive patterns in `step`); you do not need to handle this case explicitly.
15. Write a function `bigstep :: Term → Term` that implements big-step semantics (Exercise 3.5.17). Like `eval`, it fails with a Haskell exception on stuck terms.
16. Write a datatype `Type` with the types `Bool` and `Nat` (Section 8.2).
17. Write a function `typeof :: Term → Type` implementing the typing rules in Figures 8-1 and 8-2. On ill-typed terms it fails with a Haskell exception (non-exhaustive patterns).
18. Add a constructor for `max` to the `Term` datatype and extend `step` and `typeof` with the rules in items 8 and 9.